Safety-driven Optimization of Edge Inference in Autonomous Systems to Minimize Cloud ML Invocations

Abstract

Edge-Cloud hybrid control leverages fast but less accurate edge models with slower, high-accuracy cloud DNNs. Continuously invoking the cloud ensures precision but incurs high latency and communication cost. This paper proposes a safety-driven dynamic scheduling policy that adaptively decides when to request cloud inference based on the runtime state of the system. By "hitting" deadlines near safety thresholds and "skipping" them otherwise, the controller minimizes cloud calls while maintaining stable and safe control performance. We show that this adaptive strategy achieves performance comparable to always-on hybrid control but significantly reducing cloud usage, while maintaining system safety.

1 Introduction and Related Work

Background: Recent advances in deep learning have significantly reshaped control and perception capabilities in Cyber-Physical Systems (CPSs), powering applications from autonomous vehicles to industrial automation. However, embedding Deep Neural Networks (DNNs) directly into feedback control loops introduces a tension between computational demand and timing guarantees. Performing inference locally (edge-only) ensures low-latency actuation but is bounded by the limited compute and memory resources of embedded hardware. This constraint often necessitates using smaller or compressed DNNs, which may compromise estimation fidelity. On the other hand, cloud-based control provides access to powerful computational infrastructure and more accurate inference, but the associated network latency and potential communication failures can threaten system responsiveness and safety.

To bridge this trade-off, the community has turned to Split Computing (SC), wherein a DNN is partitioned between the edge and the cloud. The edge executes a lightweight local subnetwork (the "head") for preliminary inference, while the cloud runs the more complex portion (the "tail") for refined predictions. This configuration merges the low-latency advantages of edge inference with the accuracy benefits of cloud computation. Yet, the control loop becomes intertwined with the timing and reliability of communication links. Consequently, safety and stability depend not only on the controller design but also on when and how often the cloud module is invoked, which is the problem we address.

Related work: Pioneering works such as Neurosurgeon [11] and Auto-tuning [12] established foundational methods for dynamic SC partitioning. Minimizing the communication bottleneck at the split layer has been a major research focus, involving techniques such as pruning and quantization [5, 6, 13], and the use of Knowledge Distillation (KD) [8] for feature compression. KD-based approaches like Distilled Split DNNs [15] and BottleFit [16] focus on learning compact intermediate representations. These efforts are complemented by architectural frameworks such as CDE [20], *I*-SPLIT [7], Split-Et-Impera [1], and MTL-Split [3] (which can leverage sparsity [2]), that integrate compression and multi-task learning into the split-layer design. Given the increasing reliance on DNNs in safety-critical CPSs, recent work has shifted focus to the implications of SC latency and accuracy on control loop performance. To address the timing uncertainties of the platform [9], how different SC architectures impact controller design for learning-enabled CPS [4] have also been explored. The problem was formulated as that of sizing edge/cloud DNNs to maximize system safety by combining low-delay, high-uncertainty edge estimates with delayed, high-accuracy cloud estimates [23], with specific applications exploring DNN sizing for the safety of vehicular CPS [24]. These efforts often utilize the F1TENTH platform as a common benchmark for such autonomous control systems [19, 24]. The search for highly efficient architectures, such as MobileNetV3 [10], often complements these efforts.

Our contributions: Building on this line of research, a key remaining challenge is determining when the cloud should be invoked within a control loop. One possible hybrid control framework is to assume a fixed cloud invocation period, querying the cloud every k sampling cycles. While effective in reducing communication compared to continuous cloud updates, this fixed scheduling overlooks the dynamic nature of system behavior. In practice, system uncertainty, environmental disturbances, and network variations may all fluctuate over time. We propose that the optimal cloud-calling frequency should adapt accordingly.

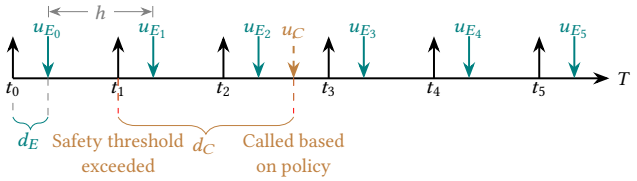


Figure 1: Timing diagram of our hybrid edge-cloud controller.

In this work, we propose a hybrid edge–cloud control strategy supporting *dynamic invocation policies*, where cloud control engagement is based on system safety criteria. Rather than invoking the cloud at a fixed interval, the cloud control is evoked only when system deviation exceeds certain predefined threshold. We also propose several dynamic invocation strategies in which the cloud controller is triggered according to different probability-based decision rules, where the invocation probability is shaped by the current system deviation. This dynamic approach enables more efficient bandwidth utilization, improved scalability, and enhanced resilience under time-varying workloads or network congestion—without compromising control performance or safety. We term this control strategy the Cloud-Online strategy.

Figure 1 illustrates the timing relationships in the proposed Cloud-Online strategy. Sampling instants t_i occur periodically with interval h, where each sampled state is transmitted to both the edge and cloud DNNs. The green downward arrows denote edge-generated control inputs u_{E_i} received after delay d_E, while the brown arrows represent cloud-generated control inputs u_{C_i}, received after a longer delay d_C. To support fine-grained adaptation,

we further develop a family of probabilistic and deterministic invocation policies for the cloud input. These include Bernoulli-based triggers, max-miss guarded strategies, deviation-aware logistic and exponential probability models, and a fully deterministic threshold rule. Each mechanism allows the system to trade communication cost against control precision while retaining explicit guarantees on worst-case cloud-update spacing or deviation tolerance. They allow balancing responsiveness and global accuracy in real time.

Using an F1Tenth autonomous racing platform, we show that the proposed Cloud-Online strategies reduce cloud usage substantially while preserving or even improving trajectory accuracy compared to both Cloud-Only and Cloud-Offline baselines. The key insight is that missing deadlines intentionally, when done according to safety-driven criteria, can make learning-enabled CPS both more efficient and more resilient.

Paper organization: The remainder of this paper is organized as follows. Section 2 summarizes relevant control-theoretic background and system-level safety metrics. Section 3 details the dynamic hybrid control architecture and its invocation policy. Section 5 presents experimental results demonstrating the efficacy of adaptive scheduling. Finally, Section 6 concludes with remarks on future extensions toward fully self-optimizing SC controllers.

2 Background on Control Theory

We consider CPSs modeled as Linear Time-Invariant (LTI) systems, whose plant dynamics evolve according to constant system matrices and linear differential equations.

Continuous-Time Systems. The continuous-time state-space representation is given by $\dot{x}(t) = A \cdot x(t) + B \cdot u(t)$, where $x(t) \in \mathbb{R}^n$ denotes the state vector, $u(t) \in \mathbb{R}^m$ the control input, and $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$ the system matrices defining the dynamics of the plant.

Discrete-Time Systems. Since digital controllers operate on sampled measurements, it is standard to discretize the continuous-time dynamics using a sampling interval h. The resulting discrete-time system evolves as $x[k+1] = \Phi \cdot x[k] + \Gamma \cdot u[k]$, where $\Phi = e^{Ah}$ and $\Gamma = \int_0^h e^{As} B\, ds$ capture the sampled dynamics. Here, $x[k]$ represents the state at time $t = k \cdot h$, and $u[k]$ the control input held over that sampling interval.

Time-Delayed Systems. In realistic deployments, delays arise from computation, sensing, and communication. Introducing a total delay d into the discrete-time controller modifies the state update equation to $x[k+1] = \Phi \cdot x[k] + \Gamma_1(d) \cdot u[k-1] + \Gamma_0(d) \cdot u[k]$, where $\Gamma_0(d) = \int_0^{h-d} e^{As} B \cdot ds$ and $\Gamma_1(d) = \int_{h-d}^{h} e^{As} B \cdot ds$. This representation can be recast into an augmented delay-free system, $z[k+1] = \Phi_{\text{aug}} \cdot z[k] + \Gamma_{\text{aug}} \cdot u[k]$, with augmented state $z[k] = \begin{bmatrix} x[k] \\ u[k-1] \end{bmatrix}$, $\Phi_{\text{aug}} = \begin{bmatrix} \Phi & \Gamma_1(d) \\ 0 & 0 \end{bmatrix}$, $\Gamma_{\text{aug}} = \begin{bmatrix} \Gamma_0(d) \\ \mathbf{I} \end{bmatrix}$. This formulation enables the analysis and design of controllers for systems subject to inference or communication delays using standard discrete-time techniques.

State-Feedback Controllers. A linear state-feedback controller takes the form $u[k] = -K \cdot x[k] + F \cdot r$, where r is the reference state, $K \in \mathbb{R}^{m\times n}$ is the feedback gain, and $F = 1 \,/\, \left((\mathbf{I} - \Phi + \Gamma K)^{-1} \Gamma\right)$ is the feedforward term ensuring reference tracking. The gain K is typically designed using the Linear–Quadratic Regulator (LQR) method, which minimizes the infinite-horizon quadratic cost $J = \sum_{k=0}^{\infty} (x[k]'Qx[k] + u[k]'Ru[k])$, where Q and R weight state deviation and control effort, respectively.

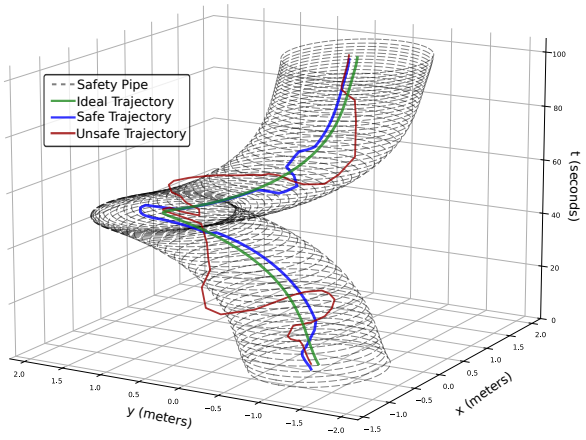


Figure 2: A safety pipe as a quantifiable system-level safety property.

2.1 System-Level Safety Metrics

In addition to stability and performance, CPSs must satisfy strict *safety* constraints to ensure that state trajectories remain within permissible operating regions at all times. Although stability guarantees eventual convergence to a reference, transient behavior must also remain sufficiently bounded to prevent unsafe conditions. Quantitative safety metrics therefore enable adaptive scheduling strategies in which cloud inference or supplemental computation is invoked only when the system exhibits elevated risk.

Following [9], one such metric is the Euclidean distance between the instantaneous system state and a prescribed safety boundary, referred to as the *safety pipe*. The safety pipe represents a tubular safe region surrounding the nominal (ideal) trajectory of the system. As illustrated in Figure 2, the nominal trajectory forms the centerline of the pipe, while a sequence of concentric rings of fixed radius outlines a cylindrical envelope in three-dimensional space. Any non-ideal system trajectory that remains within this cylindrical envelope is considered safe, as it indicates that disturbances or modeling errors have not pushed the system beyond acceptable bounds. Conversely, if the trajectory exits the pipe at any point, it is classified as unsafe, signaling a violation of the permissible operating region.

Operationally, the safety pipe metric measures how close the system is to breaching this boundary. When the minimum distance to the pipe wall falls below a specified margin, the controller must immediately transition to a conservative fallback policy, such as mandatory cloud inference, until the deviation decreases to an acceptable level. This provides a direct, interpretable pathwise criterion for runtime safety assurance.

A complementary metric assesses the consistency between the continuous-time trajectory and its discrete-time approximation, defined as the value computed as Root Mean Square Error (RMSE) = $\sqrt{\frac{1}{H} \sum_{k=0}^{H} \left(\frac{\|x(k\cdot h) - x[k]\|}{\|x[k]\|} \right)^2}$, where H is the time horizon, $x(k \cdot h)$

denotes the ideal continuous-time state at $t = k \cdot h$, $x[k]$ denotes the corresponding non-ideal discrete-time state at iteration k, and $\|\cdot\|$ is the Euclidean norm. Lower Root Mean Square Error (RMSE) values signify closer alignment with the nominal continuous-time evolution. Within our dynamic invocation framework, these metrics determine when additional cloud inference should be initiated, thereby linking safety and cloud utilization decisions.

3 Edge-Cloud Hybrid Control

Consider a plant whose (continuous-time) dynamics are $\dot{x}(t) = A_c x(t) + B_c u(t)$, $x(t) \in \mathbb{R}^n$, $u(t) \in \mathbb{R}^m$, sampled with period $h > 0$. Sensor data obtained at the start of each sampling period is processed in parallel by a small, fast network $\texttt{DNN}_E$ on the edge and a larger, more accurate network $\texttt{DNN}_C$ in the cloud. The computation (and transfer) delays for the edge and cloud networks are denoted d_E and d_C, respectively, with the operating assumption $0 \le d_E < d_C < h$, (*i.e.*, the edge output is available earlier than the cloud output and both are available within one sampling period).

At the beginning of the k-th sampling interval (which we identify with the time interval $[k \cdot h, (k+1) \cdot h)$), three piecewise-constant control values are applied to the plant in succession:

(1) From $t = k \cdot h$ to $t = k \cdot h + d_E$: the held cloud inference from the *previous* interval, $u_C[k-1]$, is applied.
(2) From $t = k \cdot h + d_E$ to $t = k \cdot h + d_C$: the fresh edge inference $u_E[k]$ is applied.
(3) From $t = k \cdot h + d_C$ to $t = (k+1) \cdot h$: the fresh cloud inference $u_C[k]$ (if available) is applied.

When a cloud call is missed at time k the quantity $u_C[k]$ is not available and the controller continues to use the previously held $u_C[k-1]$. Let $\Phi = e^{A_c h}$. The state at the next sampling instant can be written by integrating the continuous dynamics over the three sub-intervals. Thus the discrete update is $x[k+1] = \Phi \cdot x[k] + \Gamma_1(d_E, d_C) \cdot u_E[k] + \Gamma_2(d_C) \cdot u_C[k] + \Gamma_3(d_E) \cdot u_C[k-1]$, where the three input-effect matrices are $\Gamma_1(d_E, d_C) = \int_{d_E}^{d_C} e^{A_c s} B_c \cdot ds$, $\Gamma_2(d_C) = \int_{d_C}^{h} e^{A_c s} B_c \cdot ds$, and $\Gamma_3(d_E) = \int_0^{d_E} e^{A_c s} B_c \cdot ds$.

3.1 Augmented State Representation

To express the dynamics in standard linear state–space form we augment the plant state with the most recently held cloud output. We define $z[k] = \begin{bmatrix} x[k] \\ u_C[k-1] \end{bmatrix} \in \mathbb{R}^{n+m}$ and $u[k] = \begin{bmatrix} u_E[k] \\ u_C[k] \end{bmatrix} \in \mathbb{R}^{2m}$. Using this notation the state update equation becomes:

$$z[k+1] = \underbrace{\begin{bmatrix} \Phi & \Gamma_3(d_E) \\ 0 & 0 \end{bmatrix}}_{\Phi_{\text{aug}}} \cdot z[k] + \underbrace{\begin{bmatrix} \Gamma_1(d_E, d_C) & \Gamma_2(d_C) \\ 0 & I_m \end{bmatrix}}_{\Gamma_{\text{aug}}} \cdot u[k],$$

and the controller implements a state-feedback law of the form $u[k] = K \cdot z[k]$, with $K \in \mathbb{R}^{2m \times (n+m)}$.

3.2 LQR Design

We design the gain K using discrete-time LQR on the augmented system. Let $Q \ge 0$ and $R > 0$ be weighting matrices of compatible dimensions. The infinite-horizon quadratic cost is $J = \sum_{k=0}^{\infty} \left(z[k]^\top Q z[k] + u[k]^\top R u[k] \right)$, and the discrete-time algebraic Riccati equation for the closed-loop pair ($\Phi = \Phi_{\text{aug}}, \Gamma = \Gamma_{\text{aug}}$) is $P = \Phi^\top P \Phi - \Phi^\top P \Gamma \cdot (\Gamma^\top P \Gamma + R)^{-1} \cdot \Gamma^\top P \Phi + Q$. Solving the above yields P, and the optimal state-feedback gain is $K = (\Gamma^\top P \Gamma + R)^{-1} \cdot \Gamma^\top P \Phi$.

4 Safe Hybrid Controller Design

We consider a family of call–scheduling strategies that trade cloud usage (expense and latency) against control fidelity. Let $\rho \in [0, 1]$ denote the Cloud Call Ratio (CCR), *i.e.*, the long-run fraction of sampling instants at which the $\texttt{DNN}_C$ is queried for an updated inference. Let t index sampling instants, let $u_E(t)$ denote the control inferred locally by $\texttt{DNN}_E$ at time t, and let $u_{C\,\text{hold}}(t)$ denote the most recent cloud inference available to the controller. When the cloud is not queried at t, the controller applies the previously held $u_{C\,\text{hold}}(t)$. We now describe the five strategies we studied.

To motivate these strategies, we note that they address different facets of the core accuracy–latency trade-off inherent in hybrid edge–cloud control. The always-hit baseline provides an upper bound on achievable performance but ignores opportunities to reduce cloud usage when the system is safely converging. Simple Bernoulli sampling introduces controllable rate reduction, while its max-miss–guarded variant adds deterministic protection against long stretches of missed updates—important because they can amplify deviation growth under delayed dynamics. Deviation-aware probabilistic policies further refine cloud utilization by increasing invocation likelihood precisely when the discrepancy between edge and held-cloud estimates becomes large, thereby steering computation toward the most safety-critical moments. Finally, the deterministic threshold rule represents the sharpest form of deviation-driven control, triggering cloud inference only when the measured mismatch exceeds a fixed tolerance. Together, these strategies span a progression from rate-based to safety- and state-aware invocation, offering a structured way to explore accuracy–cost trade-offs in the hybrid controller.

Strategy 1 — Always–hit (baseline): The baseline hybrid controller queries $\texttt{DNN}_C$ at every sampling instant and therefore achieves $\rho = 1$. The controller always applies the freshest cloud inference $u_C(t) = u_{C\,\text{cloud}}(t)$, and no output holding occurs. This strategy provides the upper bound (highest accuracy, highest cost) for comparison.

Strategy 2 — Bernoulli (memoryless probabilistic hits): At each sampling instant t, we perform an independent Bernoulli trial with parameter $p \in [0, 1]$. A successful trial ("hit") triggers a cloud call and updates $u_{C\,\text{hold}}(t) = u_{C\,\text{cloud}}(t)$. A failure ("miss") results in $u_{C\,\text{hold}}(t) = u_{C\,\text{hold}}(t-1)$. In expectation, $\mathbb{E}[\rho] = p$, and the inter-hit gaps follow a geometric distribution with mean $1/p$. This strategy is simple and tunable but admits unbounded streaks of misses.

Strategy 3 — Bernoulli with max*-miss guard:* To prevent arbitrarily long periods without cloud updates, we enforce a limit $m \in \mathbb{Z}_{\ge 0}$ on the number of consecutive misses. Bernoulli sampling with parameter p is executed as in Strategy 2, but if no hit has occurred for m consecutive samples, we *force* a hit on the next sample.

Let L be the number of samples between successive hits. A renewal analysis yields

$$\mathbb{E}[L] = \sum_{k=0}^{m-1} (1-p)^k p(k+1) + (1-p)^m (m+1) = \frac{1 - (1-p)^{m+1}}{p},$$

and therefore the steady-state cloud call ratio is $\rho = \frac{1}{\mathbb{E}[L]} = \frac{p}{1-(1-p)^{m+1}}$. As $m \to \infty$ we recover $\rho = p$ (Strategy 2), while $m = 0$ yields $\rho = 1$. The guard enforces a deterministic upper bound of $m + 1$ samples between cloud hits.

Strategy 4 — Deviation-aware probabilistic hits: Rather than using time-only randomization, the hit probability becomes a function of the deviation

$$d(t) = \|u_E(t) - u_{C\,\mathrm{hold}}(t)\|.$$

At each sampling instant, a Bernoulli trial is executed with a deviation-dependent hit probability $p_{\mathrm{hit}}(d(t))$. These strategies concentrate cloud usage on time instants where the discrepancy between local and held cloud inferences is largest.

(i) Logistic:

$$p_{\mathrm{hit}}(d;\beta,\kappa) = \sigma(\kappa(d-\beta)), \qquad \sigma(z) = \frac{1}{1+e^{-z}},$$

where β sets the midpoint (activation threshold), and $\kappa > 0$ controls steepness. As $\kappa \to \infty$, the policy approaches a deterministic threshold at $d = \beta$.

(ii) Exponential:

$$p_{\mathrm{hit}}(d;\beta,\lambda) = 1 - \exp\left(-\lambda \cdot [d-\beta]_+\right), \qquad [z]_+ = \max(z, 0),$$

where β is an activation threshold and $\lambda > 0$ determines how sharply the probability rises once $d > \beta$.

(iii) Piecewise-linear ramp: A third deviation-aware stochastic policy employs a bounded linear ramp. Let $0 \le \beta_{\mathrm{low}} < \beta_{\mathrm{high}} \le 1$ and define

$$p_{\mathrm{hit}}(d) = \begin{cases} 0, & d \le \beta_{\mathrm{low}}\, p, \\ \dfrac{d - \beta_{\mathrm{low}}\, p}{(\beta_{\mathrm{high}} - \beta_{\mathrm{low}})\, p}, & \beta_{\mathrm{low}}\, p < d < \beta_{\mathrm{high}}\, p, \\ 1, & d \ge \beta_{\mathrm{high}}\, p. \end{cases}$$

This formulation increases the hit probability linearly as the deviation approaches the safety boundary and saturates deterministically beyond it, yielding a simple and interpretable stochastic approximation of a hard threshold.

At each sampling instant, the controller computes $p_{\mathrm{hit}}(d(t))$ and samples a Bernoulli random variable to determine whether a cloud call is issued. A *max*-miss guard from Strategy 3 may optionally be combined with any of the above deviation-aware probabilistic rules to enforce a worst-case inter-hit bound.

Strategy 5 — Deterministic threshold (deviation-triggered): This non-stochastic rule invokes the cloud when the deviation exceeds a user-specified threshold θ:

$$\text{call cloud at time } t \iff d(t) > \theta.$$

The resulting CCR is $\rho = \Pr\{d(t) > \theta\}$. This strategy is recovered as the infinite-slope limit of the logistic or exponential models and provides a simple, interpretable guarantee on the maximum tolerated discrepancy. Empirically, this policy offered the strongest accuracy–cost trade-off after tuning θ.

4.1 Design tradeoffs, safety mechanisms, and utilization control

The space of invocation policies presents several complementary tradeoffs that guide practical deployment.

Smoothness, robustness, and responsiveness. Compared to deterministic thresholding, deviation-aware probabilistic strategies (Strategy 4) yield smoother behavior near the activation boundary. Hard thresholds can induce miss–hit oscillations in both deviation and processor utilization, whereas stochastic policies soften this transition, often reducing limit-cycle amplitudes and utilization bursts. Moreover, smooth probability mappings improve robustness to noise and modeling uncertainty by mitigating threshold chatter.

Worst-case guarantees and tail risk. A key limitation of probabilistic policies is the introduction of tail risk: even near the safety boundary, there remains a non-zero probability of a miss. This complicates formal safety assurance and weakens worst-case guarantees. To mitigate this effect, hard safety overrides may be enforced, such as deterministically triggering a cloud call whenever $d(t) \ge p$, or conservatively at $d(t) \ge \rho p$ for some $\rho < 1$. Alternatively, a probability floor $p_{\min}$ can be imposed near the boundary to bound miss likelihood.

Processor utilization and sliding-window constraints. Because stochastic invocation introduces randomness in cloud usage, instantaneous processor demand can fluctuate. To maintain predictable utilization, a token-bucket mechanism may be employed. Each cloud hit consumes one token; tokens are replenished at rate r, and the bucket capacity B bounds allowable burstiness. Over any sliding window of duration T, the expected number of hits satisfies

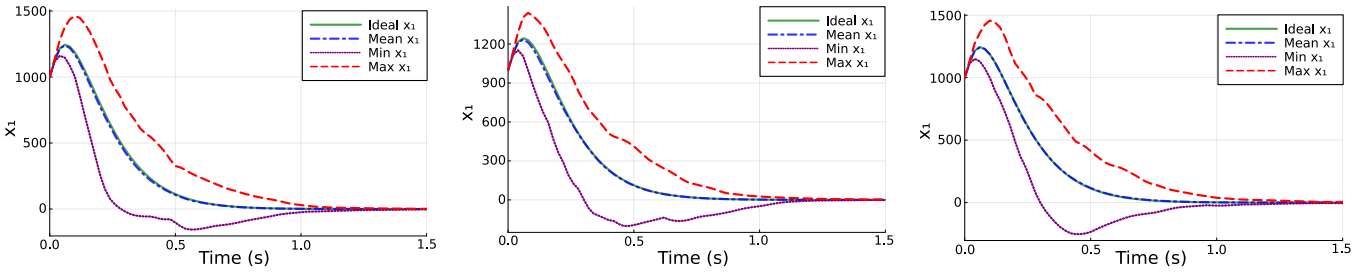
$$\mathbb{E}\left[\sum_{k=t}^{t+T-1} a_k\right] = \sum_{k=t}^{t+T-1} p_{\mathrm{hit}}(d_k) \le r\,T + B,$$

where $a_k \in \{0, 1\}$ denotes whether a hit occurs at time k. This constraint ensures that expected processor utilization remains within budget despite stochastic scheduling decisions.

Composition and coordination. Probabilistic policies can be combined with *max*-miss guards, hard overrides, or utilization budgets to balance responsiveness with safety. In multi-task settings, parameter heterogeneity and randomization can further reduce synchronization effects, spreading computational demand in time and lowering the risk of coincident deadline hits.

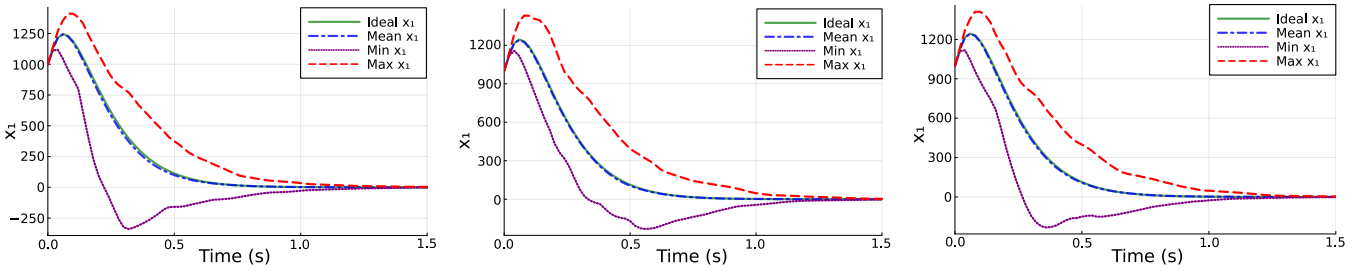
Parameter tuning. A staged workflow remains effective. Data collected under simple Bernoulli or guarded strategies can be used to calibrate deviation-aware parameters $(\beta, \kappa, \lambda, \beta_{\mathrm{low}}, \beta_{\mathrm{high}})$ before transitioning to deterministic or probabilistic deviation-triggered rules. When learning-based tuning is employed, the resulting deployment remains lightweight, requiring storage of only a small set of scalar parameters.

In summary, these strategies span the spectrum from simple rate control (Strategy 2), to bounded blackout protection (Strategy 3), to deviation-aware stochastic shaping (Strategy 4), and finally to deterministic thresholding (Strategy 5). While probabilistic policies offer smoother behavior and finer safety–efficiency tuning,



(a) All cloud calls (p=1) **(b) Exponential (λ=0.2, β=1; m=2)** **(c) Threshold (θ=1.6)**

Figure 3: System dynamics obtained with different strategies to reduce the number of cloud calls.



(a) Simple random (p=0.6) **(b) Simple random with 1 max miss (p=0.6, m=1)** **(c) Simple random with 2 max misses (p=0.6, m=2)**

Figure 4: System dynamics obtained using simple random strategy with different values of allowed max misses.

they trade deterministic guarantees for probabilistic ones and therefore benefit from complementary safety and utilization constraints.

5 Experiments

Here, we compare baseline invocation strategies against our proposed adaptive approaches and analyze their impact on control accuracy, safety-relevant deviation, and cloud resource usage.

Objectives: Our experimental study is to answer the following questions: **(1)** How does the hybrid strategy perform relative to purely stochastic or threshold-based invocation policies? **(2)** What benefits arise from enforcing maximum consecutive missed cloud inferences ("max misses")? **(3)** How do invocation strategies trade control accuracy versus cloud call ratio? **(4)** To what extent do stochastic strategies (simple random, logistic, exponential) approximate the performance of the hybrid scheme at lower cloud usage?

Setup: We simulate the closed-loop system using the F1TENTH plant model and the hybrid edge–cloud controller described in Section 3. The controller employs an LQR gain designed over the augmented delay-aware dynamics (Section 3.2). All invocation strategies are evaluated under identical trajectories, initial conditions, and noise realizations.

Hardware platforms: The edge device is modeled after an NVIDIA Jetson Nano [17] with a peak compute throughput of 472 GFLOP/s. The cloud platform is modeled after an NVIDIA GeForce RTX 5060 [18] with a compute capability of 614 TFLOP/s. A round-trip transmission delay of 0.1 s is assumed, consistent with 5G communication latency measurements [14].

Neural network models: The edge network $\texttt{DNN}_E$ is instantiated as `MobileNetV3_Small`, while the cloud network $\texttt{DNN}_C$ is instantiated as `EfficientNet_B6`, as in [21]. The edge model provides fast but low-accuracy inference with an error rate of 42.53% and an inference delay of $d_E = 0.02$ s. The cloud model provides higher accuracy (error rate 21.08%) but, including the communication latency, has a larger inference delay of $d_C = 0.10$ s. Both networks are modeled as introducing zero-mean Gaussian estimation noise, as $u_E = u_{\text{true}} + \mathcal{N}(\mu{=}0, \sigma_E{=}0.7)$ and $u_C = u_{\text{true}} + \mathcal{N}(\mu{=}0, \sigma_C{=}0.3)$, with $\sigma_E > \sigma_C$ reflecting the relative accuracies.

Plant model: The plant is the standard state-space model of an F1TENTH autonomous racing vehicle [19]. The state vector is $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, where x_1 is vehicle velocity and x_2 is yaw angle. The control input $u = \begin{bmatrix} u_{\text{acc}} \\ u_{\text{steer}} \end{bmatrix}$ consists of acceleration and steering commands. The vehicle begins at a non-zero initial velocity, and the controller aims to bring the vehicle to rest while maintaining yaw stability. The sampling period is set to h, and $d_E < d_C < h$.

Invocation strategies: We evaluate a family of cloud-invocation mechanisms that trade accuracy against cloud usage, each corresponding to the strategies formalized in Section 4. As an upper bound, a **hybrid** strategy (based on Strategy 1 from Section 4) queries the cloud at every sampling instant. To reduce invocation rate, a **simple random** policy (Strategy 2) calls the cloud according to a Bernoulli trial with probability p=0.6, while its safety-enhanced counterpart, a **simple random with max misses** (Strategy 3), enforces a limit of $m \in \{1, 2\}$ consecutive missed cloud calls to guarantee a bounded inter-hit gap.

Moving to state-aware probabilistic rules, the **logistic random** strategy (a realization of Strategy 4) shapes the hit probability via a logistic function with parameters κ=0.02, β=1, and its guarded variant, **logistic + max misses** augments this with m=2. Similarly,

Table 1: Summary of RMSE statistics and cloud call ratios for all invocation strategies.

Strategy to reduce cloud invocations	mean RMSE	std. RMSE	min. RMSE	max. RMSE	Cloud Call Ratio
Hybrid (all cloud calls hit)	63.06	33.68	8.35	304.59	100.00%
Simple random (p=0.6)	66.37	36.19	8.67	328.95	60.09%
Simple random with max misses (p=0.6, m=1)	63.72	33.77	7.22	283.91	71.44%
Simple random with max misses (p=0.6, m=2)	64.63	34.35	6.49	301.83	64.14%
Logistic random (β=1, κ=0.02)	66.17	34.76	8.41	319.43	63.04%
Logistic random with max misses (β=1, κ=0.02, m=2)	65.44	34.28	6.81	334.59	67.25%
Exponential random (β=1, λ=0.2)	64.07	34.07	8.97	286.00	62.05%
Exponential random (β=1, λ=0.2, m=2)	63.37	33.61	9.08	310.32	70.57%
Threshold (θ=1.6)	62.38	32.89	7.30	269.43	65.10%

the **exponential random** strategy (also Strategy 4) uses an exponential deviation-to-probability mapping with λ=0.2, β=1, and its **exponential + max misses** variant introduces a guard of m=2 to limit worst-case blackout intervals. Finally, reflecting the deterministic limit of the deviation-aware family, the **threshold-based** strategy (Strategy 5 in Section 4) triggers cloud inference whenever the deviation exceeds $\theta = 1.6$. All strategies operate on identical input sequences, delays, neural network uncertainty models, and controller parameters. This ensures fair comparison, and they are evaluated jointly in terms of safety-related deviation (via RMSE) and cloud-side computational demand (via invocation ratio), thereby enabling a joint evaluation of accuracy and resource efficiency.

Methodology: For each strategy, we simulate the closed-loop plant evolution over a fixed horizon. At each timestep, the invocation strategy determines whether the cloud estimate u_C is computed or whether the controller relies solely on the edge estimate u_E . The resulting control input is applied to the plant model, generating the next state.

In order to characterize the performance of each of our considered strategies, we compute the root-mean-square deviation between the discrete trajectory and its continuous-time analogue (Section 2.1). For each strategy we report mean, standard deviation, minimum (min.), and maximum (max.) RMSE across all runs. We also compute the fraction of timesteps at which a cloud invocation occurs (CCR).

To visually discern the control performance of the same strategies, we plot system trajectories under (i) the hybrid strategy, (ii) exponential random, and (iii) threshold-based invocation. We also plot the evolution under (i) simple random, (ii) simple random with m=1, and (iii) simple random with m=2. These plots illustrate transient behavior, control responsiveness, and the effect of delayed cloud returns. Finally, we also tabulate the numerical results (as summarized in Table 1) which report the RMSE statistics and CCRs for all strategies.

5.1 Results

Table 1 summarizes the performance of all invocation strategies. Several trends emerge. The threshold-based policy (θ=1.6) achieves the lowest mean RMSE among the non-hybrid approaches (62.38) at a moderate CCR of 65.1%, indicating that deviation-aware invocation can match or slightly outperform the always-cloud baseline by concentrating cloud inference on high-deviation segments. The hybrid controller remains the natural upper bound, obtaining a mean RMSE of 63.06 at a 100% CCR, and provides a reference point against which the efficiency of the other policies can be judged.

Within the randomized family, simple Bernoulli sampling with p=0.6 results in a mean RMSE of 66.37 and a worst-case RMSE of 328.95, reflecting the occasional long stretches of missed cloud calls inherent to memoryless sampling. Introducing a constraint on consecutive misses substantially stabilizes behavior. With m=1, mean RMSE drops to 63.72 and the maximum deviation decreases to 283.91; m=2 yields a similar effect with mean RMSE 64.63 and CCR 64.14%. These results show that bounding the inter-hit gap mitigates large transient deviations while maintaining cloud usage near the original p.

The comparison of the hybrid baseline against the CCR reduction strategies, as depicted in Figure 3, demonstrates the superior performance attainable through a state-aware cloud calling mechanism. Specifically, the Threshold strategy ($\theta = 1.6$) yielded the lowest overall mean RMSE (62.38), surpassing even the hybrid controller (mean RMSE 63.06) which utilizes 100% of cloud calls. This improved fidelity, achieved at an efficient 65.1% CCR, is visually confirmed in Figure 3c where the mean x_1 trajectory (blue dotted line) exhibits near-perfect tracking and almost completely overlaps the ideal x_1 reference. The Threshold method further provides the highest stability, registering the minimal max. RMSE (269.43) across all variants, indicating enhanced safety against transient errors. Among the temporal stochastic methods, the Exponential random strategy augmented with a maximum consecutive miss constraint of $m = 2$ proved most effective, achieving a mean RMSE of 63.37 at a 70.57% CCR. The system dynamics in Figure 3b for this configuration visibly confirm the tight tracking of the mean x_1 to the ideal x_1, validating its near-optimal performance relative to the 100% CCR baseline.

Figure 4 illustrates the critical role of temporal regularization in managing stochastic volatility, specifically the effect of the maximum consecutive misses parameter (m) on the simple random strategy ($p = 0.6$). The unconstrained simple random approach (Figure 4a) resulted in the worst performance (mean RMSE 66.37, max. RMSE 328.95) and exhibited high volatility due to statistically probable long stretches of missed cloud calls, leading to a significant spread between min x_1 and max x_1. The introduction of a stringent constraint, $m = 1$ (Figure 4b), dramatically improved control fidelity (mean glsrmse 63.72), approaching the Hybrid baseline. This improvement, however, required a substantial increase in resource usage, resulting in a 71.44% CCR. The visual system dynamics are markedly smoother and tighter than the $m = 0$ case, with the floor of min. x_1 noticeably elevated. Relaxing the constraint to $m = 2$ (Figure 4c) achieved a favorable cost-performance trade-off: the mean RMSE remained tightly controlled (64.63) while the CCR dropped

efficiently to 64.14%. This demonstrates that a moderate maximum miss constraint stabilizes the system dynamics effectively, *i.e.* maintaining the tighter min. x_1 or max. x_1 spread observed in the $m = 1$ case, without the cost penalty associated with the stricter constraint. Finally, the range of maximum RMSE values, occasionally exceeding 300 for unconstrained policies, illustrates the influence of rare sequences of missed cloud invocations. Evaluating such tail behavior is therefore essential when assessing safety. Future versions of this study will incorporate percentile-based statistics and hypothesis tests to more systematically compare strategies and quantify the robustness of observed differences.

6 Concluding Remarks and Future Work

We studied hybrid edge-cloud control for learning-enabled CPSs and evaluated a family of cloud invocation strategies for DNN-enabled perception processing that trade cloud utilization against closed-loop accuracy and safety. Our modeling framework explicitly accounts for edge and cloud inference delays via an augmented discrete-time representation and enables controller synthesis (LQR) on the resulting delay-aware dynamics. **Experimentally**, we find the state-aware invocation—especially the deterministic threshold strategy—yields the best accuracy-cost trade-off, achieving the lowest RSME while reducing cloud calls by about one third. Stochastic policies with a max-miss guard also perform well, offering practical, easily deployable alternatives that provide bounded worst-case blackout intervals and useful data for tuning deviation-triggered rules. However, some randomized schemes exhibit rare but large RSME spikes, underscoring the need to report variance- and tain-sensitive metrics when assessing CPS safety in these setups.

For future work we will: (i) incorporate an explicit cost model (monetary or energy per cloud call) to optimize a joint accuracy–cost objective, (ii) extend the invocation policy design to jointly schedule cloud calls for multiple controllers under a weakly-hard constraint [22], such as "only 2 out of 3 controllers can make a cloud call at the same instant", where the controller that is made to miss the deadline can be decided according to a fixed priority or by allowing the controller with the most recently hit deadline to be made to miss its next deadline in order to maintain the constraint, (iii) explore predictive triggers (using short-term forecasts of $d(t)$), such that the controller can compensate for lost cloud inference results due to network issues, and (iv) explore reinforcement-learning based schedulers that aim to maximize a reward function that balances both safety and cost using some tunable parameter. Finally, we plan to validate the approach on hardware-in-the-loop and multi-agent scenarios, and to investigate formal safety guarantees (*e.g.*, size of the reachable set) for the controller and invocation policy.

References

[1] Luigi Capogrosso et al. 2023. Split-Et-Impera: A Framework for the Design of Distributed Deep Learning Applications. In *International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*. IEEE.

[2] Luigi Capogrosso et al. 2024. Enhancing Split Computing and Early Exit Applications through Predefined Sparsity. In *Forum on Specification & Design Languages (FDL)*. IEEE, 1–8.

[3] Luigi Capogrosso, Enrico Fraccaroli, Samarjit Chakraborty, Franco Fummi, and Marco Cristani. 2024. MTL-Split: Multi-Task Learning for Edge Devices using Split Computing. In *Design Automation Conference (DAC)*.

[4] Luigi Capogrosso, Shengjie Xu, Enrico Fraccaroli, Marco Cristani, Franco Fummi, and Samarjit Chakraborty. 2024. Learning-Enabled CPS for Edge-Cloud Computing. In *2024 IEEE 14th International Symposium on Industrial Embedded Systems (SIES)*. IEEE. doi:10.1109/SIES62473.2024.10767956

[5] Damiano Carra and Giovanni Neglia. 2023. DNN Split Computing: Quantization and Run-Length Coding are Enough. In *Global Communications Conference (GLOBECOM)*. IEEE.

[6] Hyomin Choi and Ivan V Bajić. 2018. Deep Feature Compression for Collaborative Object Detection. In *25th International Conference on Image Processing (ICIP)*. IEEE.

[7] Federico Cunico et al. 2022. I-SPLIT: Deep Network Interpretability for Split Computing. In *International Conference on Pattern Recognition (ICPR)*. IEEE.

[8] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. 2021. Knowledge Distillation: A Survey. *International Journal of Computer Vision* 129, 6 (2021), 1789–1819.

[9] Clara Hobbs et al. 2022. Safety Analysis of Embedded Controllers Under Implementation Platform Timing Uncertainties. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4016–4027.

[10] Andrew Howard et al. 2019. Searching for MobileNetV3. In *International Conference on Computer Vision (ICCV)*.

[11] Yiping Kang et al. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. *SIGPLAN Notices* 52, 4 (2017).

[12] Guangli Li et al. 2018. Auto-tuning Neural Network Quantization Framework for Collaborative Inference Between the Cloud and Edge. In *Artificial Neural Networks and Machine Learning (ICANN)*. Springer.

[13] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and Quantization for Deep Neural Network Acceleration: A Survey. *Neurocomputing* 461 (2021), 370–403.

[14] Reiner Ludwig and Ericsson. 2022. Who cares about latency in 5G? https://www.ericsson.com/en/blog/2022/8/who-cares-about-latency-in-5g Ericsson blog, Accessed: 2025-05-18.

[15] Yoshitomo Matsubara et al. 2019. Distilled Split Deep Neural Networks for Edge-Assisted Real-Time Systems. In *Workshop on Hot Topics in Video Analytics and Intelligent Edges at Mobicom*.

[16] Yoshitomo Matsubara et al. 2022. BottleFit: Learning Compressed Representations in Deep Neural Networks for Effective and Efficient Split Computing. In *International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*.

[17] NVIDIA. 2019. NVIDIA Jetson Nano. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/ Accessed: 2025-05-18.

[18] NVIDIA. 2025. NVIDIA GeForce RTX 5060. https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5060-family/ Accessed: 2025-05-18.

[19] Matthew O'Kelly et al. 2020. F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement Learning. In *Proceedings of Machine Learning Research (PMLR)*, Vol. 123. PMLR, 77–89.

[20] Marion Sbai, Muhamad Risqi U. Saputra, Niki Trigoni, and Andrew Markham. 2021. Cut, Distil and Encode (CDE): Split Cloud-Edge Deep Inference. In *International Conference on Sensing, Communication, and Networking (SECON)*. IEEE.

[21] Shengjie Xu et al. 2024. GPU Partitioning & Neural Architecture Sizing for Safety-Driven Sensing in Autonomous Systems. In *International Conference on Assured Autonomy (ICAA)*.

[22] Shengjie Xu, Bineet Ghosh, Clara Hobbs, P. S. Thiagarajan, and Samarjit Chakraborty. 2023. Safety-Aware Flexible Schedule Synthesis for Cyber-Physical Systems using Weakly-Hard Constraints. In *2023 28th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 46–51.

[23] Tingan Zhu et al. 2025. Controllers for Edge-Cloud Cyber-Physical Systems. In *International Conference on Communication Systems and Networks (COMSNETS)*.

[24] Tingan Zhu, Mier Li, Bineet Ghosh, Samarjit Chakraborty, and Parasara Sridhar Duggirala. 2025. Safety-Driven DNN Sizing for Vehicular CPS. *IEEE Embedded Systems Letters* (2025), 1–1. doi:10.1109/LES.2025.3595839